

Atividade: Pipeline de Classificação CIFAR-10

Objetivo

Construir um **pipeline completo de classificação** para o dataset CIFAR-10 (imagens 32×32 RGB, 10 classes), incluindo **pré-processamento**, **divisão treino/validação/teste** com semente fixa, **validação para melhoria de desempenho**, **treinamento de uma CNN** e **avaliação final**. O uso de Inteligência Artificial generativa para a resolução do problema será penalizado na nota da atividade.

Configuração e Regras

- Linguagem: Python (PyTorch *ou* TensorFlow/Keras).
- Fixe a semente aleatória para reprodutibilidade:

```
random_state = 2025
```

- Divisão dos dados: **70% treino, 15% validação, 15% teste**.
- Métricas mínimas a reportar: acurácia em validação por época, acurácia em teste ao final e matriz de confusão do conjunto de teste.

Pipeline de Execução

1) Leitura e organização do dataset

Baixe/carregue o CIFAR-10 usando a API da biblioteca escolhida. Mostre: 10 amostras aleatórias (classe e miniaturas), contagem por classe e forma dos dados.

2) Pré-processamento e *data augmentation*

Normalize as imagens para [0, 1] ou padronize por canal (média e desvio do CIFAR-10).

Recomenda-se **augmentação leve** para o conjunto de treino:

```
RandomCrop(32, padding=4) RandomHorizontalFlip() ColorJitter (moderado)
```

3) Divisão dos Dados em treino/validação/teste

Realize a divisão estratificada, preservando a proporção de classes. Registre a semente (2025) na função de *split* e nos geradores de lotes. Demonstre, em tabela, o número de amostras por classe em cada partição.

4) Arquitetura sugerida de rede (CNN)

Utilize uma CNN pequena e eficiente para CIFAR-10, por exemplo:

- **Layer 1:** Conv(3→32, 3×3, padding=1) + BN + ReLU; Conv(32→32, 3×3, padding=1) + BN + ReLU; MaxPool(2×2).
- **Layer 2:** Conv(32→64, 3×3, padding=1) + BN + ReLU; Conv(64→64, 3×3, padding=1) + BN + ReLU; MaxPool(2×2).

- **Head:** Dropout(0.3); Flatten; Dense(256) + ReLU; Dropout(0.5); Dense(10) + Softmax (Keras) ou Logits + CrossEntropy (PyTorch).

5) Treinamento com validação para melhoria de desempenho

Objetivo: usar a validação para *model selection* e evitar *overfitting*. Utilize os seguintes parâmetros:

- a) **Otimizador:** SGD (lr=0.1, momentum=0.9, weight decay=5e-4) ou Adam (lr=1e-3).
- b) **Learning rate scheduling:** redução on-plateau na métrica de validação ou marcos em épocas: 60/120.
- c) **Early stopping:** pare o treino se a acurácia de validação não melhorar após 10 épocas (salve o melhor classificador).
- d) Plote *curvas treino vs. validação* comparando a acurácia e a perda.

6) Avaliação no conjunto de teste

Carregue o melhor classificador salvo (segundo validação) e reporte a acurácia final de teste, a matriz de confusão e o relatório de classificação por classe. Discuta onde o modelo erra mais e quais classes são mais confusas.

Entregáveis

- Notebook (.ipynb) com todo o pipeline, código e gráficos (curvas de treino/validação, matriz de confusão).
- Relatório completo em PDF, contendo capa, sumário, explicação de todos os conceitos empregados e análise dos resultados, bem como o papel da validação na melhoria do desempenho do classificador obtido.